SQL GROUP BY & HAVING

Mastering Aggregations, Group-Level Filtering, and Clause Execution Order

Part A: The GROUP BY Clause

Understanding Data Grouping

The `GROUP BY` clause collapses multiple rows that share the same values into summary rows. It is almost always paired with aggregate functions like `COUNT()`, `SUM()`, `AVG()`, `MIN()`, and `MAX()`.

```sql
-- Total salary and headcount by department
SELECT
    department,
    COUNT(*) AS total_employees,
    SUM(salary) AS total_payroll
FROM employees
GROUP BY department;
```

Clause Element	Rule / Purpose
`SELECT column`	Any unaggregated column in SELECT **must** be in GROUP BY.
`Multiple Columns`	Group by multiple dimensions: `GROUP BY department, job_title`
`NULL Handling`	All NULL values in the grouping column are treated as a single group.

Part B: The HAVING Clause

Filtering Aggregated Groups

The `HAVING` clause was introduced because the `WHERE` keyword cannot be used with aggregate functions. `HAVING` evaluates conditions on summarized groups *after* aggregation.

```sql
-- Departments with more than 5 employees and average salary > 60,000
SELECT
    department,
    COUNT(*) AS emp_count,
    AVG(salary) AS avg_salary
FROM employees
GROUP BY department
HAVING COUNT(*) > 5 AND AVG(salary) > 60000;
```

Part C: WHERE vs HAVING Comparison

Feature	WHERE	HAVING
Applies to	Individual rows before grouping	Aggregated groups after grouping
Aggregate functions?	❌ No (e.g., cannot do WHERE SUM(...) > 100)	✅ Yes (e.g., HAVING COUNT(*) >= 2)
Query position	Before `GROUP BY`	After `GROUP BY`
Performance	High (filters out rows early before aggregation)	Evaluates only after groups are formed

SQL Clause Execution Order

1. `FROM` & `JOIN`: Retrieve tables and link rows
2. `WHERE`: Filter raw rows
3. `GROUP BY`: Aggregate rows into groups
4. `HAVING`: Filter groups
5. `SELECT`: Compute columns and aggregate expressions
6. `ORDER BY`: Sort final result set
7. `LIMIT / OFFSET`: Restrict row count

Part D: Common Pitfalls & Best Practices

⚠️ Pitfall 1: Selecting non-grouped columns

Selecting columns not wrapped in an aggregate function and not in `GROUP BY` causes an error in standard SQL (`ONLY_FULL_GROUP_BY` in MySQL).

⚡ Tip 2: Filter rows early with WHERE

Always place non-aggregate conditions in `WHERE` rather than `HAVING` to reduce the number of rows processed during grouping.

```sql
-- RECOMMENDED: Filter active employees before grouping
SELECT department, AVG(salary)
FROM employees
WHERE status = 'ACTIVE'
GROUP BY department
HAVING AVG(salary) > 50000;
```

Part E: 20 Practice Questions

Test Your GROUP BY & HAVING Skills

1. Count the total number of employees in each department.
2. Find the average salary for each job title.
3. Find the minimum and maximum salary in each department.
4. Calculate total sales revenue by each product category.
5. Find departments having more than 10 employees.
6. Find departments where the average salary is greater than $75,000.
7. Find total sales per region for regions with total sales exceeding $500,000.
8. Group employees by department and gender, showing the headcount for each group.
9. Find job titles that have an average bonus less than $2,000.
10. Find departments with at least 3 employees who have a salary above $50,000.
11. Count orders placed per customer, showing only customers who placed 5 or more orders.
12. Find the highest earning employee salary per department.
13. Find the average order value per country where the minimum order value is greater than $100.
14. Display the count of products supplied by each supplier for active suppliers only.
15. Find departments where the total payroll is between $200,000 and $1,000,000.
16. Find store branches where the maximum discount offered was under 15%.
17. Find years from the `hire_date` with more than 15 new hires.
18. Calculate total units sold per product, sorted from highest units to lowest.
19. Find customers whose average purchase quantity exceeds 10 items.
20. Find departments where the difference between max and min salary is greater than $40,000.